creates a migration called *yyyyMMddhhmmss*_create_discounts.rb:

```
depot> bin/rails generate model discount
    invoke  active_record
»   create    db/migrate/20221207133549_create_discounts.rb
    create    app/models/discount.rb
    invoke    test_unit
    create      test/models/discount_test.rb
    create      test/fixtures/discounts.yml
```

- You can also generate a migration on its own.

```
depot> bin/rails generate migration add_price_column
    invoke  active_record
»   create    db/migrate/20221207133814_add_price_column.rb
```

Later, starting in *Anatomy of a Migration*, we'll see what goes in the migration files. But for now, let's jump ahead a little in the workflow and see how to run migrations.

Running Migrations

Migrations are run using the db:migrate Rake task:

```
depot> bin/rails db:migrate
```

To see what happens next, let's dive down into the internals of Rails.

The migration code maintains a table called schema_migrations inside every Rails database. This table has just one column, called version, and it will have one row per successfully applied migration.

When you run bin/rails db:migrate, the task first looks for the schema_migrations table. If it doesn't yet exist, it'll be created.

The migration code then looks at the migration files in db/migrate and skips from consideration any that have a version number (the leading digits in the filename) that's already in the database. It then proceeds to apply the remainder of the migrations, creating a row in the schema_migrations table for each.